🪟 Game States & Menus in MLX Python

> Part of the MLX Python Documentation. Function details are in the MLX Reference.

Why This Topic Is Important

One of the first things students build in MLX projects is a set of screens: main menu, settings, gameplay, game over, theme selection.

It looks simple, but beginners quickly run into:

- multiple windows opening by accident
- images overlapping, or old menus not disappearing
- hooks that stop working after changing screen
- crashes when reloading assets
- duplicated rendering loops and frozen windows

Almost all of these come from the same cause: **the screens are not managed as states**.

⚠️ The Key Idea: One Window, Many States

MLX can create multiple windows, but for a menu system you almost always want **only one**.

❌ Wrong idea:

```
Main Menu Window → Gameplay Window → Settings Window → Theme Window
```

✅ Recommended:

```
One MLX window + one variable saying which screen is active
```

This is how most games work internally.

Why multiple windows cause trouble

- each window needs its own hooks, so they get duplicated or forgotten
- keyboard focus moves between windows
- windows and images must be destroyed manually, or they leak
- it is hard to know which window should be redrawn

🧠 What Is a Game State?

A game state is simply **which screen the game is currently showing**:

```
MENU · PLAYING · SETTINGS · THEME_SELECTION · GAME_OVER
```

Changing screen means changing the state — **not** opening a new window.

```
Program starts
     ↓
   MENU ── press "1" ──▶ PLAYING ── press ESC ──▶ MENU
     │
     └── press "2" ──▶ THEME MENU
```

The window never changes. Only what is drawn in it changes.

📦 Basic State Architecture

A small class keeps all the state in one place:

```
        class GameState:
            def __init__(self):
                self.screen = "MENU"
                self.selected_option = 0
                self.theme = "DARK"
                self.needs_redraw = True

        state = GameState()
```

> Use instance attributes (`self.x` in `__init__`), not class attributes. Class attributes are shared and are easy to modify by mistake.

🔄 Rendering: the Loop Hook

You don't write a `while running:` loop in MLX — `mlx_loop` already is the loop. You register **one** loop hook, and it draws whatever the current state is:

```
        def on_loop(param):
            if not state.needs_redraw:
                return
            mlx.mlx_clear_window(mlx_ptr, win_ptr)
            if state.screen == "MENU":
                draw_menu()
            elif state.screen == "PLAYING":
                draw_game()
            elif state.screen == "SETTINGS":
                draw_settings()
            state.needs_redraw = False

        mlx.mlx_loop_hook(mlx_ptr, on_loop, None)
        mlx.mlx_loop(mlx_ptr)
```

Each screen has its own draw function, so the project stays organized.

> The `needs_redraw` flag avoids redrawing hundreds of times per second when nothing changed. Set it to `True` every time the state changes.

⌨️ Input: One Hook, Dispatched by State

Register the key hook **once**, at startup. Inside it, decide what the key does based on the current screen:

```
        KEY_ESC, KEY_UP, KEY_DOWN, KEY_ENTER = 65307, 65362, 65364, 65293

        def on_key(key, param):
            if state.screen == "MENU":
                menu_controls(key)
            elif state.screen == "PLAYING":
                gameplay_controls(key)
            state.needs_redraw = True

        mlx.mlx_key_hook(win_ptr, on_key, None)
```

This is how the same arrow keys can move the selection in the menu and move the player in the game.

> Don't re-register hooks every time the screen changes. If hooks "stop working" after changing screen, it's usually because a new window was created or a hook was replaced.

🧼 Clearing Old Screens

A very common problem: **old images stay on screen**.

MLX doesn't know about your screens — it only draws what you tell it to. When the state changes, redraw everything, in this order:

1. clear the window (`mlx_clear_window`) or draw a full-size background
2. draw the map / main content
3. draw the player and entities
4. draw the UI on top

🎯 Menu Navigation

A menu needs three things:

- **state:** `selected_option`
- **input:** up, down, enter
- **renderer:** draws the options and highlights the selected one

```
        OPTIONS = ["PLAY", "SETTINGS", "EXIT"]

        def menu_controls(key):
            if key == KEY_UP:
                state.selected_option = (state.selected_option - 1) % len(OPTIONS)
            elif key == KEY_DOWN:
                state.selected_option = (state.selected_option + 1) % len(OPTIONS)
            elif key == KEY_ENTER:
                choice = OPTIONS[state.selected_option]
                if choice == "PLAY":
                    state.screen = "PLAYING"
                elif choice == "SETTINGS":
                    state.screen = "SETTINGS"
                elif choice == "EXIT":
                    mlx.mlx_loop_exit(mlx_ptr)
```

The `%` makes the selection wrap around in both directions.

To highlight the selected option you can use a different text color, an arrow (`> PLAY`), a highlighted image or an animated selector:

```
        def draw_menu():
            mlx.mlx_put_image_to_window(mlx_ptr, win_ptr, background, 0, 0)
            for i, label in enumerate(OPTIONS):
                color = 0xFFFFFF00 if i == state.selected_option else 0xFFFFFFFF
                prefix = "> " if i == state.selected_option else "  "
                mlx.mlx_string_put(mlx_ptr, win_ptr, 100, 200 + i * 30, color, prefix +
label)
```

> Find any key code by printing it inside the hook. See the key table in the reference.

🎨 Theme Switching

Themes (light, dark, gothic…) are just another piece of state:

```
        if state.theme == "DARK":
            wall_texture = dark_wall
```

Load all theme images once at startup and keep them in a dictionary. Loading them again on every switch is slow, and forgetting to `mlx_destroy_image` the old ones leaks memory.

🔄 Transitions Between Screens

Fades, slides, zooms and vortex effects are all "fake" animations: you redraw several frames in a row, changing positions or colors a bit each time. The simplest way is a `TRANSITION` state with a frame counter that the loop hook advances until it switches to the next screen.

🧱 Suggested Project Structure

```
render/
├── menu.py
├── game.py
├── settings.py
├── themes.py
└── ui.py
```

One file per screen, plus shared UI helpers.

🧠 Real Example: A Maze Project

```
MENU → GENERATE MAZE → GAMEPLAY → BFS VISUALIZATION → PLAYER MODE → END SCREEN
```

Every screen reuses the same window and the same MLX instance. Only the drawing, the meaning of the keys and the displayed assets change.

🔥 Common Problems

Symptom	Usual cause
old menu still visible	screen not cleared / background not redrawn
player moves while in the menu	key hook doesn't check the state
a second window appears	`mlx_new_window` called on screen change
hooks stop working	hooks re-registered or attached to a new window
slow theme switch / memory growth	assets reloaded instead of loaded once
window seems frozen	heavy work inside a hook blocks the loop

✅ Final Recommendation

- ✅ one window
- ✅ one state object
- ✅ one loop hook and one key hook, both dispatching on the state
- ✅ one draw function per screen
- ✅ redraw the whole screen when the state changes